DA2303 — Algorithms for Data Science

Tier	A
Phase (order)	2
Do this first	DA1300, DA1004
Estimated effort	12-14 weeks
Unlocks	DA3406, DA2403, DA3409

Roadmap

1. Complexity, order notation, master theorem, NP-hardness.
2. Divide and conquer (FFT); dynamic programming (LCS, Viterbi, backprop view).
3. Randomised algorithms and graph algorithms.

Video courses (all links checked on YouTube on 2026-10-22)

🎓 Main course: follow it start to end

- Programming, Data Structures and Algorithms using Python_Prof. Madhavan Mukund (84 videos). IIT Madras BS *Programming, Data Structures and Algorithms using Python* (Madhavan Mukund).

🎥 Other full lecture series (pick one if the main one doesn't click, or for topics it skips)

- MIT 6.006 Introduction to Algorithms, Spring 2020 (32 videos). MIT 6.006 *Introduction to Algorithms* (Spring 2020, official OCW).

⚡ Short videos for intuition and revision

- Algorithms (84 videos). Abdul Bari *Algorithms*: whiteboard explanations of each classic algorithm.

Study plan (week by week)

Lecture numbers are positions in the main playlist above. Click a number to open that video.

Week	Lectures	What to do
1	1–12	Python recap, exceptions, classes, timing code, why efficiency matters.
2	13–21	Complexity analysis, searching, selection/insertion/merge sort. Implement and time each sort.
3	22–30	Quicksort, lists vs arrays, dictionaries.
4	31–38	Graphs, BFS, DFS, DAGs, topological sort, longest path. Solve 5 graph problems on LeetCode.

Week	Lectures	What to do
5	39–45	Dijkstra, Bellman-Ford, Floyd-Warshall, Prim, Kruskal.
6	46–51	Union-find, priority queues, heaps, binary search trees, AVL trees. Implement a heap and an AVL tree.
7	52–60	Greedy (interval scheduling, lateness, Huffman), divide and conquer (inversions, closest pair, Karatsuba, quickselect).
8	61–66	Dynamic programming: memoisation, grid paths, LCS, edit distance, matrix-chain. Solve 10 DP problems.
9	67–73	String matching: Boyer-Moore, Rabin-Karp, automata, KMP, tries, regex.
10	74–84	Linear programming, network flows, reductions, P vs NP; then the summary lectures for revision.

▼ **All lectures in *Programming, Data Structures and Algorithms using Python_Prof. Madhavan Mukund* (84 videos)**

1. 1.1 : Introduction to Jupyter Notebooks and Google Colab
2. 1.2 : Implementation of Python Codes - Part 01
3. 1.3 : Python Recap - I
4. 1.4 : Python Recap - II
5. 1.5 : Python Recap - III
6. 1.6 : Exception handling
7. 1.7 : Classes and Objects
8. 1.8 : Implementation of Python Codes - Part 02
9. 1.9 : Timing Our Code
10. 1.10 : Implementation of Python Codes - Part 03
11. 1.11 : Why Efficiency Matters?
12. 1.12 : Implementation of Python Codes - Part 04
13. 2.1 : Analysis of Algorithms
14. 2.2 : Comparing Orders of Magnitude
15. 2.3 : Calculating Complexity - Examples
16. 2.4 : Searching in a List
17. 2.5 : Selection Sort
18. 2.6 : Inserting Sort
19. 2.7 : Merge Sort
20. 2.8 : Analysis of Merge Sort
21. 2.9 : Implementation of Searching and Sorting Algorithms
22. 3.1 : Quicksort
23. 3.2 : Analysis of Quicksort
24. 3.3 : Implementation of Quicksort Algorithm
25. 3.4 : Concluding Remarks on Sorting Algorithms

26. 3.5 : Difference Between Lists and Arrays (Theory)
27. 3.6 : Designing a Flexible List and Operations on the Same
28. 3.7 : Implementation of Lists in Python
29. 3.8 : Implementation of Dictionaries in Python
30. 3.9 : Difference Between Lists and Arrays (Implementation)
31. 4.1 : Introduction to Graphs
32. 4.2 : Representing Graphs
33. 4.3 : Breadth First Search (BFS)
34. 4.4 : Depth First Search (DFS)
35. 4.5 : Applications of BFS and DFS
36. 4.6 : Introduction to Directed Acyclic Graph (DAG)
37. 4.7 : Topological Sorting
38. 4.8 : Longest Path in DAGs
39. 5.1 : Shortest Paths in Weighted Graphs
40. 5.2 : Single Sourse Shortest Paths (Dijkstra's Algorithm)
41. 5.3 : Single Sourse Shortest Paths with Negative Weights (Bellman-Ford Algorithm)
42. 5.4 : All Pairs Shortest Paths (Floyd-Warshall Algorithm)
43. 5.5 : Minimum Cost Spanning Trees
44. 5.6 : Minimum Cost Spanning Trees (Prim's Algorithm)
45. 5.7 : Minimum Cost Spanning Trees (Kruskal's Algorithm)
46. 6.1 : Union-Find Data Structure
47. 6.2 : Priority Queues
48. 6.3 : Heaps
49. 6.4 : Using Heaps in Algorithms
50. 6.5 : Search Trees
51. 7.1 : Balanced Search Trees
52. 7.2 : Greedy Algorithms-Interval Scheduling
53. 7.3 : Greedy Algorithms-Minimizing Lateness
54. 7.4 : Greedy Algorithms-Huffman Coding
55. 8.1 : Divide and Conquer-Counting Inversions
56. 8.2 : Divide and Conquer-Closest Pair of Points
57. 8.3 : Divide and Conquer-Integer Multiplication
58. 8.4 : Divide and Conquer-Recursion Trees
59. 8.5 : Divide and Conquer-Quick Select
60. 8.6 : Implementation of Quick Select and Fast Select Algorithms
61. 9.1 : Dynamic Programming
62. 9.2 : Memoization
63. 9.3 : Grid Paths
64. 9.4 : Common Subwords and Subsequences
65. 9.5 : Edit Distance
66. 9.6 : Matrix Multiplication

67. 10.1 : String Matching
68. 10.2 : String Matching: Boyer-Moore Algorithm
69. 10.3 : String Matching: Rabin-Karp Algorithm
70. 10.4 : String Matching Using Automata
71. 10.5 : String Matching: Knuth-Morris-Pratt Algorithm
72. 10.6 : String Matching-Tries
73. 10.7 : String Matching-Regular Expresssions
74. 11.1 : Linear Programming
75. 11.2 : Linear Programming-Production Planning
76. 11.3 : Linear Programming-Bandwidth Allocation
77. 11.4 : Network Flows
78. 11.5 : Reductions
79. 11.6 : Intractability-Checking Algorithms
80. 11.7 : Intractability-P and NP
81. 12.1 : Summary of Weeks 1 to 3
82. 12.2 : Summary of Weeks 4 to 6
83. 12.3 : Summary of Weeks 7 to 9
84. 12.4 : Summary of Weeks 10 to 11

Books, notes and websites

- ✅ NPTEL *Design and Analysis of Algorithms* (Mukund, CMI)
- MIT 6.006 / 6.046
- Book: CLRS (course text); Motwani & Raghavan (randomised)
- Jeff Erickson, *Algorithms* (free)

Practice / proof of understanding

Solve 50 problems (DP, graphs, divide and conquer), implement Viterbi for a small HMM.

What your IITM course sheet says

From the PDF: DA2303

Official description: The students should be able to 1. Identify if a given algorithmic problem has a structure that makes it solvable. 2. Systematically analyse the complexity class of algorithms, and grasp concepts of computational hardness, i.e. NP completeness etc. 3. Identify the conditions for applying a divide-conquer, greedy, dynamic programming paradigms to problems, 4. Use the above-mentioned paradigms to develop algorithms for specific problems. 5. Deconstruct data-science problems into their fundamental computational components and build algorithms for the same.

Official syllabus (trimmed):

> Course Contents: 1. Basics. (1 week) Computational complexity, order notation, master theorem, NP-hardness, worst-case and average case complexity. 2. Basic Paradigms (3 weeks) Incremental design (insertion sort), decremental design (GCD, factorial). Divide and Conquer: Recurrences, master theorem, merge-sort, Fast-Fourier transform, 3. Dynamic Programming (3 weeks) Optimal substructure,, Recurrences, Invariants, Memoization, Recursion without memoization, Tabulation. Examples: Longest common subsequence, Viterbi algorithm for HMM inference, Backpropagation for gradient computation in Neural networks. 4. Greedy Methods (3 weeks) Greedy choice, local optimality. Examples where it is optimal: Huffman encoding, Dijkstra's shortest path. Other examples: Gradient descent, sub-modular maximization. 5. Advanced Topics (3 weeks) Continuous optimization algorithms: gradient descent, convexity, eigen-value prob- lems. Randomized algorithms: Sampling using MCMC, approximate SVD, matrix multiplication, random projections, locality sensitive hashing.

Official textbooks: 1. Cormen, Leiserson, Rivest and Stein. Introduction to Algorithms by MIT Press, Third Edition, 2009. 2. Christopher Bishop. Pattern Recognition and Machine Learning. Springer. 3. Rajeev Motwani and Prabhakar Raghavan. Randomized Algorithms. Cambridge University Press.

Official references: 1. Cormen, Leiserson, Rivest and Stein. Introduction to Algorithms by MIT Press, Third Edition, 2009. 2. Christopher Bishop. Pattern Recognition and Machine Learning. Springer. 3. Rajeev Motwani and Prabhakar Raghavan. Randomized Algorithms. Cambridge University Press.